

StarUI Platform — Architecture Guide

How the pieces fit together, for engineers landing on the codebase

2026-05-17

Contents

Foreword	2
1. Repository at a glance	2
1.1 The layer model	2
1.2 The framework matrix	4
1.3 Selected import dependencies	5
2. Runtime — the seam between OpenFin and browser	5
3. Services & platform	7
3.1 ConfigService — dual-mode (Dexie + REST)	7
3.2 How MarketsGrid profiles are stored	9
3.3 DataServices — one connection per provider, many consumers	10
3.4 Template resolution — {{name.key}} then [token]	11
4. Profile lifecycle	12
5. MarketsGrid — the flagship widget	13
5.1 Composition	13
5.2 The module pipeline	14
5.3 Storage	15
6. Hosting in OpenFin	15
6.1 The boot sequence	15
6.2 <HostedMarketsGrid> — the consolidated hosting wrapper	16
6.3 Per-view active profile	18
6.4 “Save Tab As...” — surviving workspace restore	18
7. App compositions	20
8. Extension points	21
8.1 Adding a new grid module	21
8.2 Adding a new data-provider transport	21
8.3 Adding a new tool window	21
8.4 Adding a new admin action	21
8.5 Adding a new framework-agnostic service	21
8.6 Replacing the runtime	22
9. StarUI MCP — scaffolding new apps	22
9.1 What the server does	23
9.2 Why this design	23

9.3 Status and roadmap	23
10. Operations / dev workflow	24
11. Where to look first	24

Foreword

This guide is the orientation document for engineers landing on the StarUI Platform monorepo at `/Users/develop/wfh/starui/`. It's a *technical* document, not a marketing pitch — the goal is to make the codebase navigable on day one.

It is intentionally **complementary** to two other documents:

- [ARCHITECTURE.md](#) — the canonical layer-model reference with import-boundary rules.
- [FEATURE_INVENTORY.md](#) — the reconciled feature catalogue, mapping every shipped capability to its source.

This document focuses on **interaction** — *how* the layers talk to each other, *what* the SharedWorker is doing under the hood, *why* profiles round-trip through a single AppConfigRow, and so on. Where it overlaps with the other two, it does so with diagrams and call-flow narrative.

Every diagram in the guide was generated from the Graphviz `.dot` sources under `docs/architecture-figures/`, which are checked into the repo so they can be updated alongside the code.

1. Repository at a glance

The monorepo currently produces **29 packages** plus **6 runnable apps**. Packages sit in three top-level framework buckets — `shared/`, `react/`, and `angular/` — and are arranged by **role** inside each bucket (foundation, runtime, services, hosts, providers, widgets, tools, platform shells).

The platform's central design choice is the **runtime/framework matrix**: every capability is layered so a vanilla TypeScript core can be reused unchanged across React and Angular hosts, and across OpenFin and pure-browser runtimes. Each package fits into one cell of that matrix.

1.1 The layer model

A package may import only from its own layer or layers below it.

Reading the diagram top-down:

1. **Apps** (`apps/demo-react`, `apps/markets-ui-react-reference`, etc.) compose the libraries below them but never get imported by anything else.
2. **Tools / dev UIs** are operator-facing surfaces (Config Browser, Workspace Setup, Config Editor) that sit *above* the production widgets they administer.
3. **Widgets & shells** are the user-facing components that mount inside applications — `<MarketsGrid>`, `<HostedMarketsGrid>`, the v2 Data Provider Editor, the Angular dock configurator, etc.
4. **Hosts / providers / SDK** are framework-specific glue: `<HostWrapper>`, `<ConfigServiceProvider>`, `<DataServicesProvider>`, the Widget SDK.
5. **Services & platform** is where the vanilla TypeScript core lives along with the framework-agnostic services (config, data, OpenFin shell, component hosting).
6. **Runtime** abstracts identity / theme / lifecycle behind a single `RuntimePort` interface. The same hooks work in OpenFin and browser contexts because they call only the interface.
7. **Foundation** is pure leaves — shapes, design tokens, icons, shadcn primitives. These import nothing except each other.

The Layer Model

A package may import only from its own layer or layers below it.

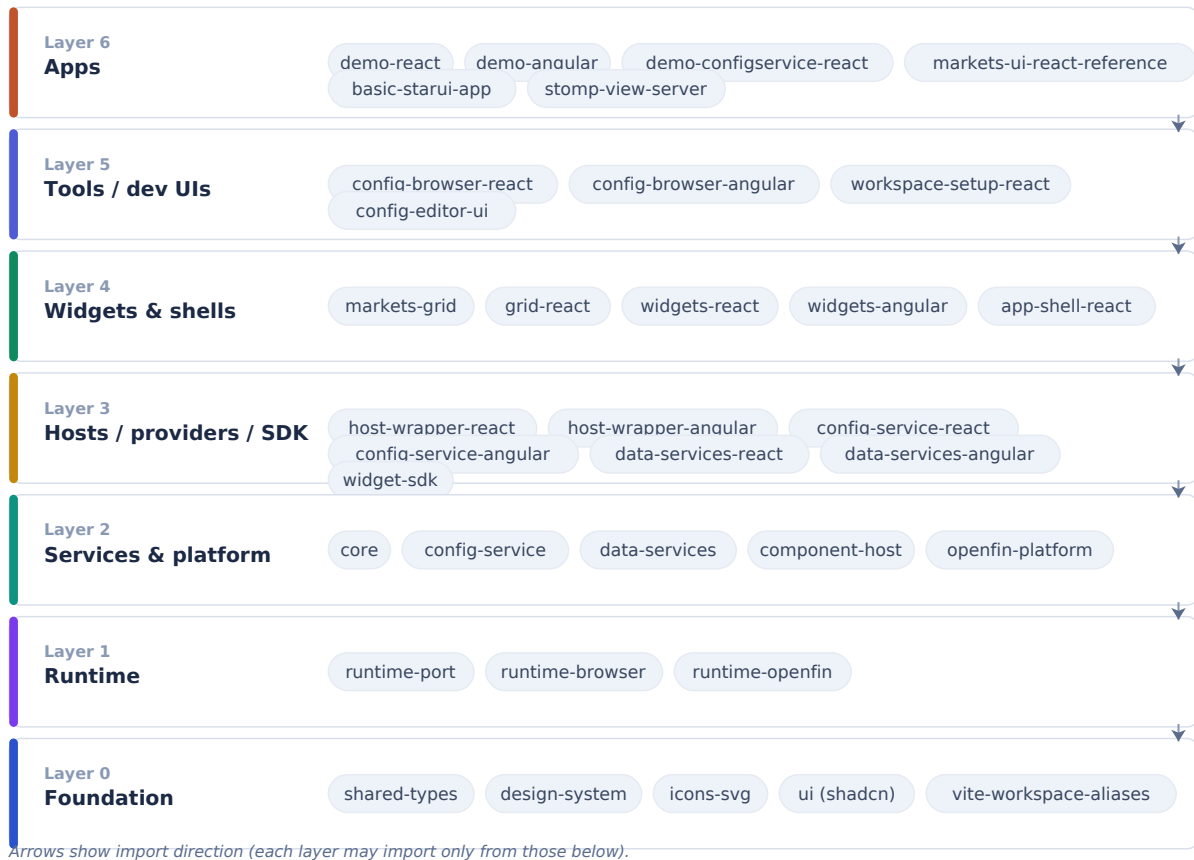


Figure 1: Layer model — 7 horizontal layers, each importing only from the layer below.

The arrows go *downward* — each layer can import from anything below it, never the reverse. This is the single rule that keeps the codebase navigable when you scale to 29 packages.

1.2 The framework matrix

Inside each layer, packages sit in one of three columns based on what they peer-depend on:

Framework Matrix

Three columns (peer-dep), six role rows.

	shared/ peer: vanilla TS	react/ peer: react	angular/ peer: @angular/*
Foundation	shared-types design-system icons-svg vite-workspace-aliases	ui (shadcn) — singleton (no -react suffix)	— DS-themed PrimeNG via downstream tools
Runtime	runtime-port (interface) runtime-browser runtime-openfin	— uses port interface	— uses port interface
Services / Platform	core config-service data-services component-host openfin-platform	widget-sdk (singleton)	— uses shared
Hosts / Providers	— no shared host	host-wrapper-react config-service-react data-services-react app-shell-react	host-wrapper-angular config-service-angular data-services-angular
Widgets & Shells	— none	markets-grid grid-react widgets-react (+ hosted/)	widgets-angular (DockConfigurator + DataProviderEditor only)
Tools / Dev UIs	— none	config-browser-react config-editor-ui workspace-setup-react	config-browser-angular

Figure 2: Framework matrix — vanilla TS, React-only, and Angular-only columns at each layer.

A few observations:

- **Foundation has no Angular twin** — shared-types, design-system, icons-svg, and vite-workspace-aliases are framework-agnostic. ui is React-only because shadcn is a React project; Angular gets PrimeNG via design-system/primeng instead.
- **Runtime is vanilla TS** — runtime-port, runtime-browser, and runtime-openfin all sit in shared/. React and Angular consume them through their respective host wrappers.
- **Services & platform are mostly vanilla TS** — core, config-service, data-services, component-host, and openfin-platform are all in shared/. The Widget SDK is the one React-singleton in this layer.
- **Hosts and providers come in pairs** — host-wrapper-react / host-wrapper-angular, config-service-react / config-service-angular, data-services-react / data-services-angular. Each pair is the framework-shaped face of the same vanilla TS service.
- **Widgets are React-led** — markets-grid, grid-react, widgets-react form the production composition for React; widgets-angular is catching up with just the DockConfigurator and DataProviderEditor today.

1.3 Selected import dependencies

Not every dependency in the repo, but the relationships that matter when you're deciding *where* to add new code:

Five Architectural Rules

The dependencies you have to internalise. Full package list — see Figure 1.

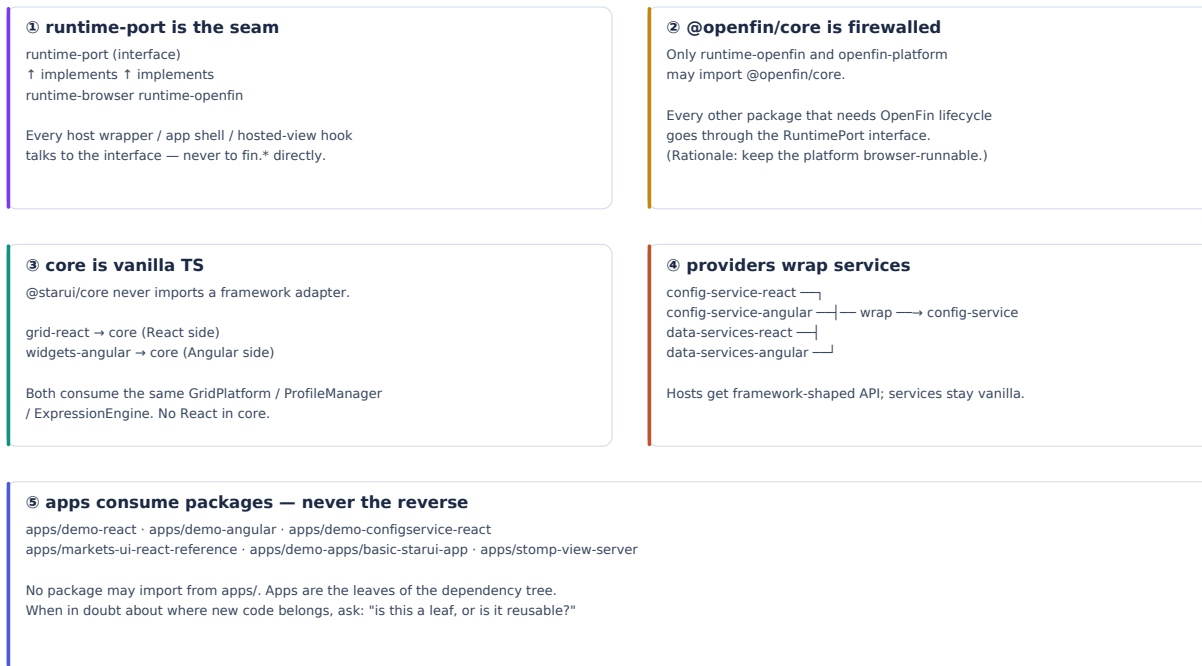


Figure 3: Selected package dependencies — bottom layers consumed by upper ones.

Three patterns to internalise:

1. **core never imports a framework adapter.** It's vanilla TS that grid-react (React) and widgets-angular (Angular) both consume.
2. **Only runtime-openfin and openfin-platform import from @openfin/core.** Every other package that needs OpenFin behaviour goes through the RuntimePort interface — that's how you build a feature once and have it work in browser dev *and* inside the OpenFin shell.
3. **config-service is consumed both directly (by the React provider, the Angular provider, the Config Browser, the workspace-setup) and via the storage factory createConfigServiceStorage that backs MarketsGrid's storage prop.** Whenever you see a profile written to disk, the call ultimately funnels through ConfigManager.saveConfig(row).

2. Runtime — the seam between OpenFin and browser

RuntimePort is the single most important interface in the codebase. Every host wrapper, every popout, every theme listener, every workspace-save hook routes through it. If you internalise this abstraction, the rest of the architecture follows naturally.

The interface lives at packages/shared/runtime/runtime-port/. It is **pure types and constants** — no behaviour, no peer deps:

RuntimePort — one interface, two implementations

The single seam between OpenFin and a plain browser.

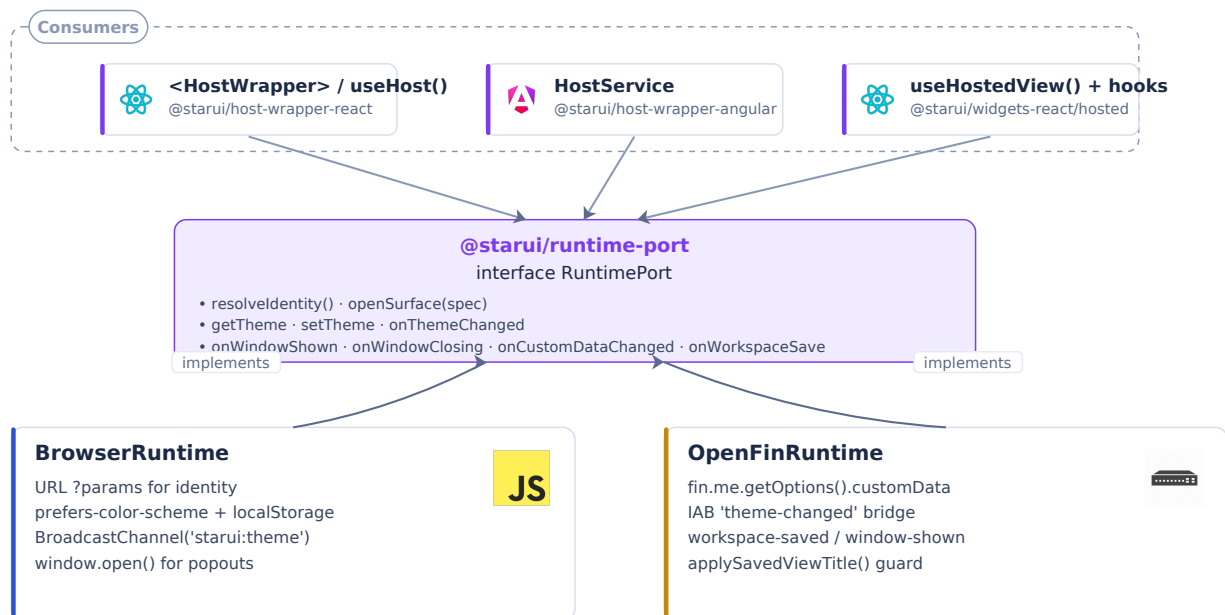


Figure 4: RuntimePort — one interface, two implementations.

```
interface RuntimePort {
  name: 'browser' | 'openfin';
  resolveIdentity(): Promise<IdentitySnapshot>;
  openSurface(spec: SurfaceSpec): Promise<SurfaceHandle>;
  getTheme(): Theme;
  setTheme(t: Theme): void;
  onThemeChanged(fn: (t: Theme) => void): Unsubscribe;
  onWindowShown(fn: () => void): Unsubscribe;
  onWindowClosing(fn: () => void | Promise<void>): Unsubscribe;
  onCustomDataChanged(fn: (cd: unknown) => void): Unsubscribe;
  onWorkspaceSave(fn: () => void | Promise<void>): Unsubscribe;
  dispose(): void;
}
```

There are exactly two implementations:

- **BrowserRuntime** — identity from URL ?params, theme from prefers-color-scheme + persisted localStorage + [data-theme], cross-tab sync via BroadcastChannel('starui:theme'), popouts via window.open, in-page modal for kind: 'modal'.
- **OpenFinRuntime** — identity from fin.me.getOptions().customData (with URL / mount-prop fallbacks), theme bridged through the OpenFin theme-changed IAB topic, workspace-saved events fanned out from the platform Channel provider, view popouts via fin.Window.create.

Two things to know about OpenFinRuntime specifically:

1. **userId is hard-pinned to LOGGED_IN_USER_ID = 'dev1'** (the single-user contract until SSO lands). customData.userId is read but ignored. The same pin is applied in runtime-browser and in every other resolution path the platform owns.
2. **applySavedViewTitle()** runs at construction time. It reads customData.savedTitle (set by the “Save Tab As...” flow — see §6.4) and reapplies it to document.title, with a 3-second MutationObserver guard that defeats the page’s mount-time useEffect that would otherwise clobber the restored title.

The constant LOGGED_IN_USER_ID = 'dev1' lives in runtime-port/src/types.ts. Replace the literal there (and the matching DEFAULT_USER_ID in registry-host-env.ts) the day SSO lands.

3. Services & platform

3.1 ConfigService — dual-mode (Dexie + REST)

ConfigService persists every piece of *user* state in the platform: MarketsGrid profiles, the dock configuration, the component registry, roles, permissions, user profiles, and the application registry. It operates in dual mode — Dexie / IndexedDB for local dev and offline caching, REST for the production server — and consumers don’t need to know which is active.

The package surfaces three primitives:

- **ConfigManager** (createConfigManager) — the live Dexie database wrapper. Owns the schema (AppConfigRow, AppRegistryRow, RoleRow, PermissionRow, UserProfileRow, PendingSyncRow), the audit-stamping pass, optimistic-lock writes, visibility filtering (isVisible(VisibilityContext)), and the profiles namespace.
- **ConfigClient** — the framework-agnostic REST-shaped API. Three implementations: LocalConfigClient (Dexie-backed), RestConfigClient (HTTP), createConfigClient (dispatcher that picks based on the supplied baseUrl).

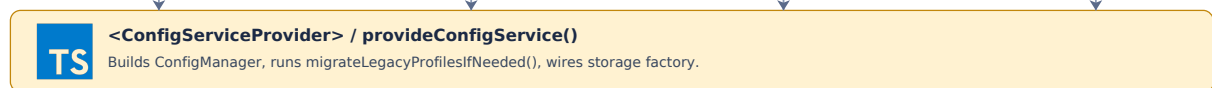
ConfigService — dual-mode (Dexie + REST)

Every piece of user state lives in one of six tables.

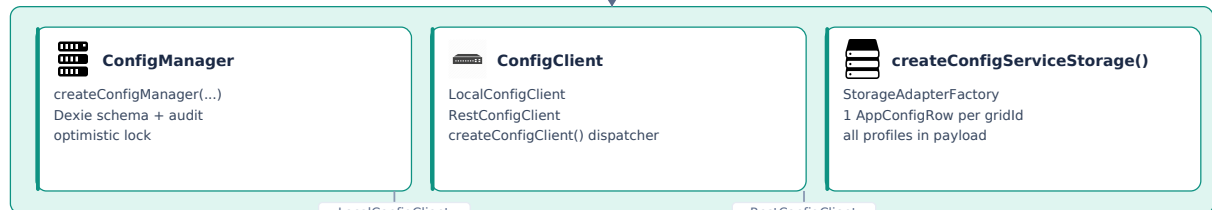
Consumers



Provider



ConfigManager + ConfigClient



Backends

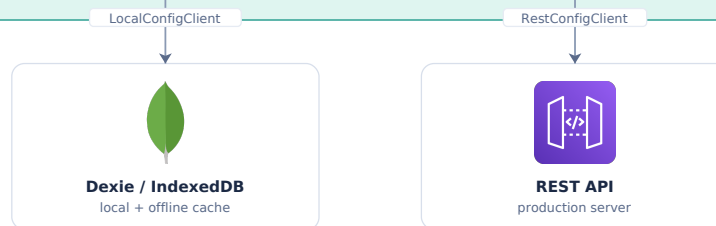


Figure 5: ConfigService — providers wrap ConfigManager + ConfigClient + storage factory.

- **createConfigServiceStorage()** — a StorageAdapterFactory that's wired into MarketsGrid's storage prop. It persists a **single AppConfigRow per (instanceId, appId, userId) scope** with all profiles bundled in the payload.

3.2 How MarketsGrid profiles are stored

The bundle row is the cleanest place to start understanding what “saving a profile” actually means:

How MarketsGrid profiles are stored

A single AppConfigRow holds the whole profile set.

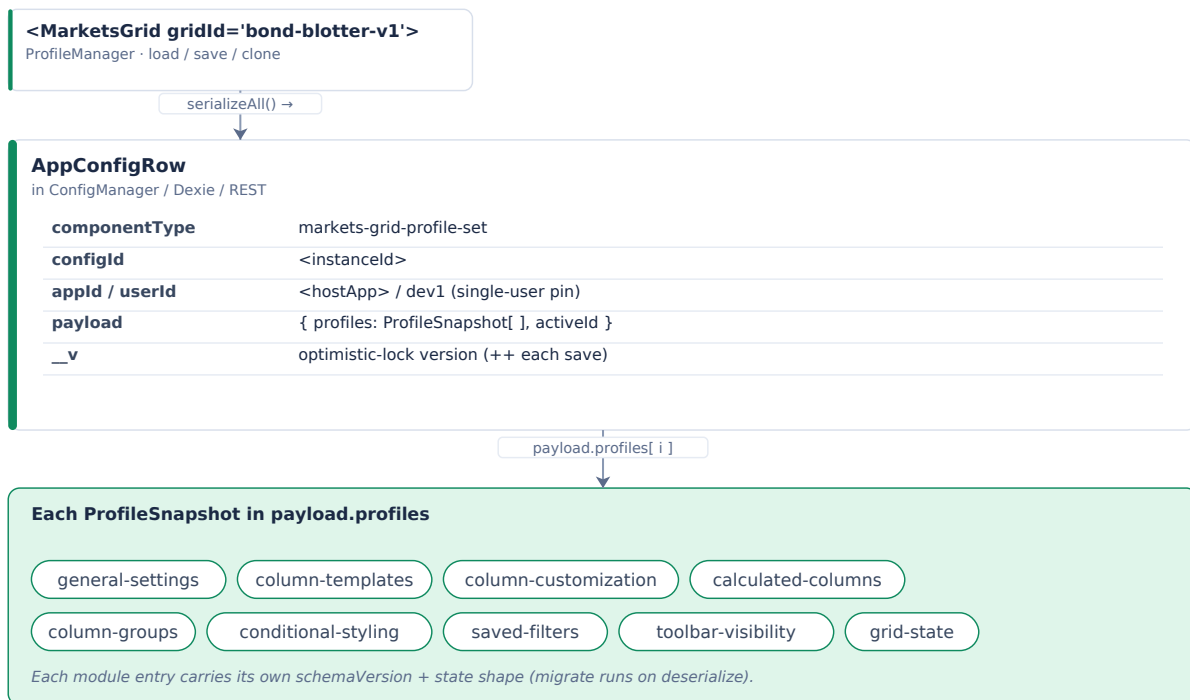


Figure 6: MarketsGrid profile-set row layout (bundle pattern).

Reading the row:

- **componentType**: "markets-grid-profile-set" is the type discriminator. There's exactly one row per grid instance.
- **configId** is the **instanceId** — so a workspace with three MarketsGrid views ends up with three rows, even if all three point at the same template.
- **appId / userId** scope the row. The single-user pin means `userId === 'dev1'` everywhere until SSO.
- **payload** is **the whole profile set**, not a single profile. Every named profile (Showcase, Risk view, User-defined #2, ...) sits inside `payload.profiles[]`. `payload.activeId` records which one is currently selected.
- **__v** is the optimistic-lock version. Each `saveConfig` write increments it; a stale write throws `ProfileSetVersionConflictError` so the UI can refresh and retry.

Inside each `ProfileSnapshot.state`, the grid serialises **per-module envelopes** — one entry per registered module, each with its own `schemaVersion` and module-defined state shape. That's how individual modules can evolve their state schemas independently (the migration runs on `deserialize`).

3.3 DataServices — one connection per provider, many consumers

@starui/data-services is the SharedWorker-backed data runtime. It exists because trading floors routinely run multiple MarketsGrid instances pointed at the same upstream data provider, and we don't want N WebSocket connections, N STOMP brokers, N REST polls. The SharedWorker multiplexes everything.

DataServices — one connection per provider

A SharedWorker multiplexes many widget subscribers onto one upstream.

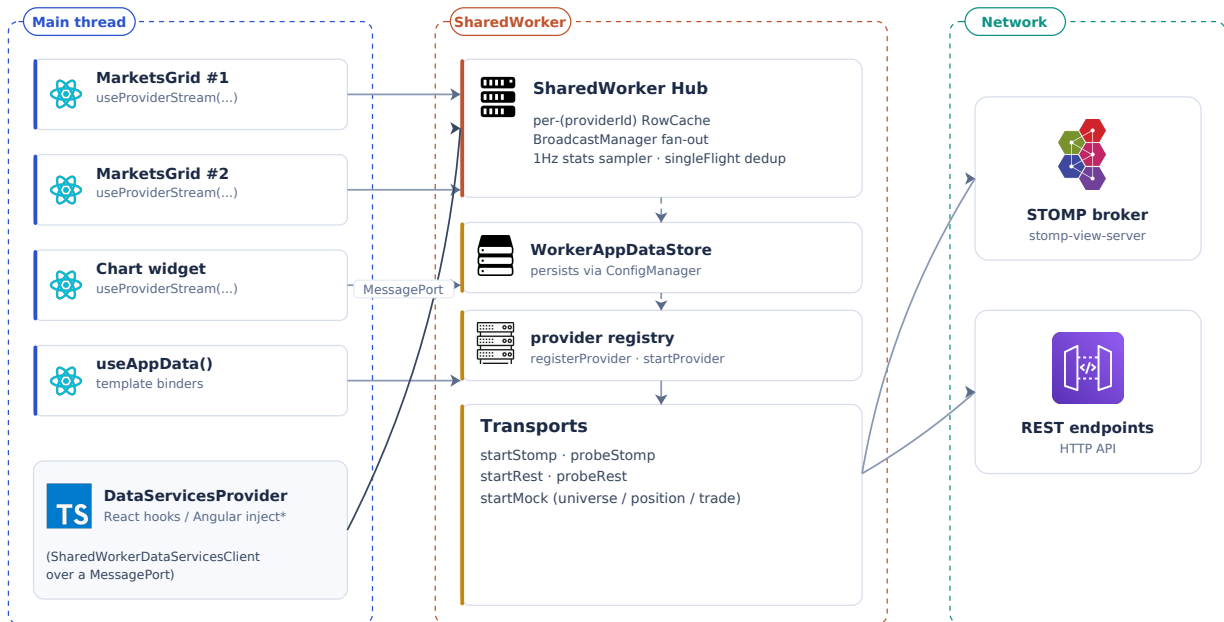


Figure 7: DataServices runtime — one SharedWorker per app, many widgets per worker.

The flow:

1. **App boot.** `bootstrapDataServices({ appName, userId, configServiceRestUrl })` constructs a `SharedWorkerDataServicesClient`, an `AppDataMirror` (sync-read + async-write proxy), and binds them to the app's `<DataServicesProvider>` (or `provideDataServices()` in Angular). The provider exposes its client to descendants.
2. **Widget mounts.** A grid calls `useProviderStream(providerId, cfg, listener)`. The hook generates a `subId`, sends an `attach({ subId, providerId, cfg, mode, extra })` request to the worker, and registers listener for incoming events.
3. **Worker side.** The hub looks up (or creates) the provider via `startProvider(cfg, emit)`. If the provider already exists, the new subscriber piggybacks on the existing connection and receives an immediate `{ rows, replace: true }` from the cache. If not, the transport (STOMP / REST / Mock) connects, populates the `RowCache` keyed by `cfg.keyColumn`, and the cache replays to every subscriber.
4. **Stats.** A 1Hz sampler on the worker emits `ProviderStats` (cache size, message count, byte size, status) to any client that subscribed in mode: `'stats'`. The `DataProviderEditor`'s Diagnostics tab is the primary consumer.
5. **AppData.** `WorkerAppDataStore` persists `useAppData()`-style key/value entries through `ConfigManager` so writes survive a reload. The `AppDataMirror` on the main thread keeps a sync-read snapshot for template resolution.

Notable rules:

- **Stream providers auto-tear-down on idle** (last subscriber leaves → release the network connection). **Keyed-resource providers do not** — AppData is pure memory and tearing it down would silently wipe state.
- **No race window between configure and subscribe.** attach is configure-or-attach: the first event back to every subscriber is always a guaranteed { replace: true, rows: [...cache] } plus the current status, so late joiners can't miss data.

3.4 Template resolution — {{name.key}} then [token]

Provider configs frequently need values that aren't known at author time — the user's id, a session-unique correlation tag, an infrastructure host. The platform supports two complementary template syntaxes that resolve at different points in the dispatch pipeline.

Template resolution — {{name.key}} then [token]

Two stages: AppData lookup on the main thread, per-attach IDs in the worker.

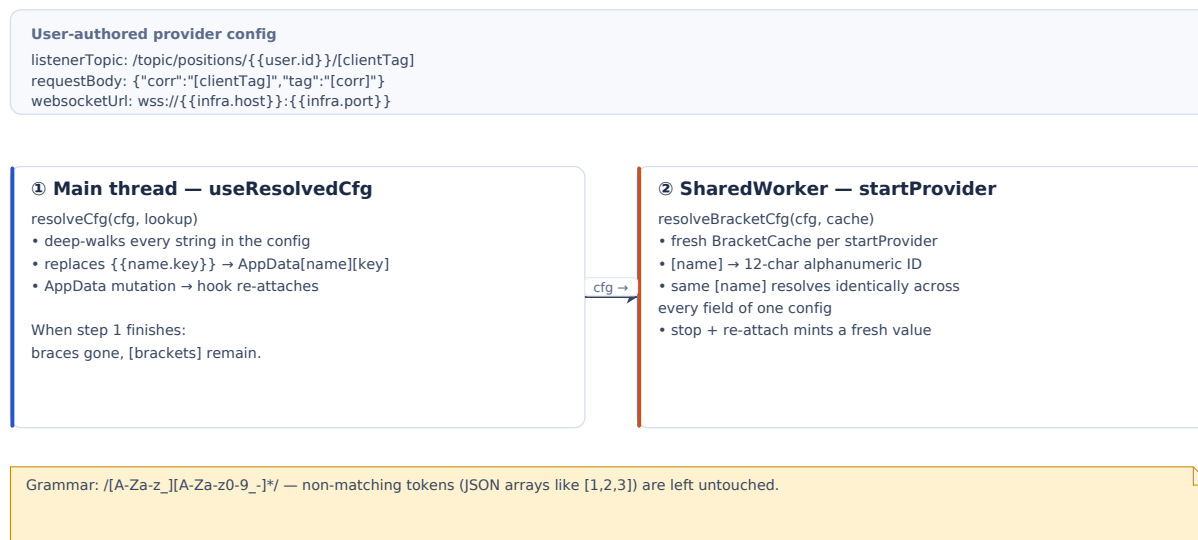


Figure 8: Template resolution — brace on main thread, bracket in worker.

- **{{name.key}}** (brace tokens) resolve **on the main thread**, inside useResolvedCfg, against an AppDataLookup snapshot served by AppDataMirror. Two important consequences:
 - The hook re-attaches whenever AppData mutates, so a value change propagates through to the worker (which calls Hub.restart(extra)) without the consumer's involvement.
 - The lookup is deterministic — same AppData, same resolved config, every time.
- **[token]** (bracket tokens) resolve **inside the worker**, in startProvider(cfg, emit), against a fresh BracketCache minted per attach. Two important consequences:
 - The same [clientTag] token in listenerTopic and in requestBody.corr resolves to **the same** 12-char alphanumeric ID, so brokers that need cross-field correlation get it for free.
 - On stop + re-attach, a fresh BracketCache is minted — perfect for reconnection correlation IDs that should be unique per session.

The grammar (/\[([A-Za-z_][A-Za-z0-9_-]*)\]/g) is restrictive on purpose: JSON arrays like [1, 2, 3] are left in place verbatim, so you can paste structured payloads into provider configs without escaping.

4. Profile lifecycle

MarketsGrid ships with `disableAutoSave: true` — the **Save button is the only write path**. This is a deliberate UX decision (profile = saved document, not live mirror) that touches several subsystems.

Profile lifecycle

Explicit Save only. The Save button is the single write path.

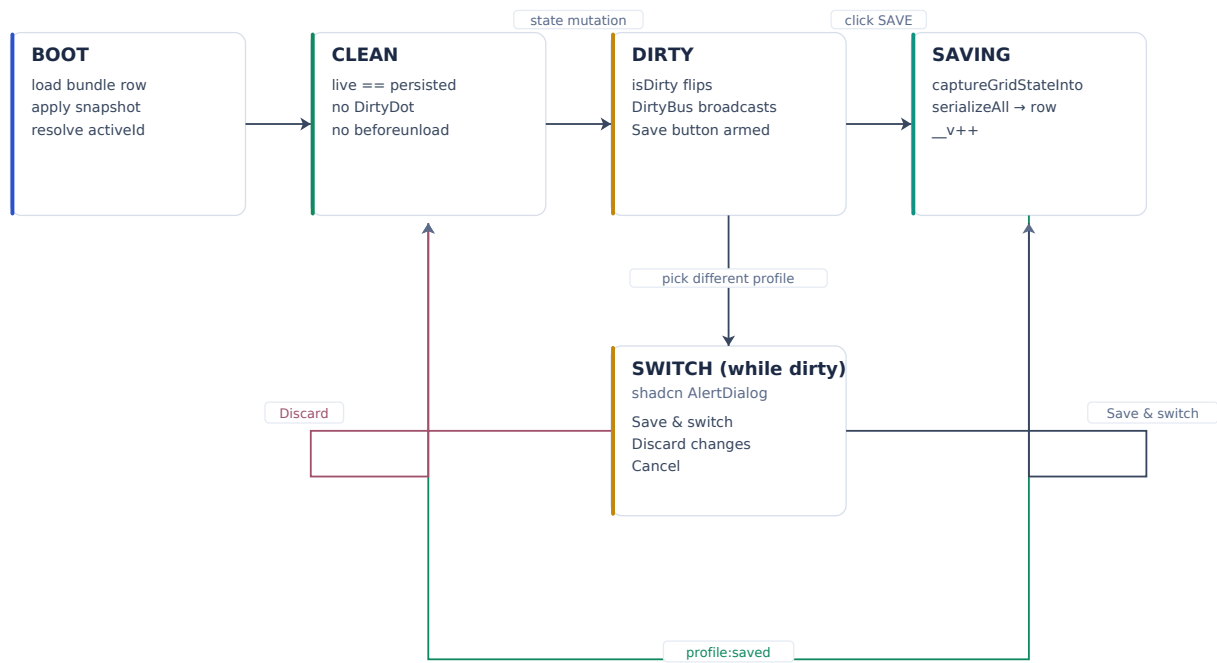


Figure 9: Profile state machine — clean / dirty / saving / switch-while-dirty.

The state machine:

- **CLEAN** — the live Zustand store matches the last persisted snapshot. The Save button is greyed; the DirtyDot is absent; no beforeunload guard is armed.
- **DIRTY** — any module-state mutation flips the internal `isDirty` flag, which broadcasts on the DirtyBus. `useDirtyCount` subscribes via `useSyncExternalStore` and updates the DIRTY=NN counter in the settings sheet header. The beforeunload listener arms; the Save button enables; the DirtyDot lights.
- **SAVING** — clicking Save triggers `captureGridStateInto(api)` so the grid-state module captures AG-Grid's native state (column order / widths / pinning / sort / filters / pagination / sidebar / focus / selection / row-group expansion / viewport anchor + `quickFilterText`). Then `serializeAll()` walks every module, builds the bundle payload, and `ConfigManager.saveConfig(row)` writes it with optimistic-lock `__v++`.
- **SWITCH while DIRTY** — picking a different profile while dirty opens a shadcn AlertDialog with three explicit choices: *Save & switch*, *Discard changes*, *Cancel*. No edit is ever silently dropped.

Other lifecycle guarantees:

- **New profile starts blank.** `createProfile` calls `resetAll()` before snapshotting; the grid-state module handles `saved: null` by calling `api.setState({})` and clearing `quickFilterText` so AG-Grid's native state is cleared too.
- **Delete safety.** `deleteProfile` cancels any pending auto-save before erasing the row, then falls back to `Default` with `skipFlush: true` so a post-delete flush can't resurrect the outgoing profile.
- **Export / Import.** Per-row Export and a footer Import flow produce `ExportedProfilePayload` blobs that round-trip through the same module migrate paths.
- **OpenFin per-view active profile.** `createOpenFinViewProfileSource()` produces an `ActiveIdSource` that reads / writes `customData.activeProfileId`. The workspace snapshot captures `customData` automatically, so duplicate views inherit and then diverge — exactly the UX traders want.

5. MarketsGrid — the flagship widget

`<MarketsGrid>` is where most engineers spend their time. It's the React composition layer that pulls together `@starui/core`, `@starui/grid-react` (modules + UI primitives + Monaco editor), and AG-Grid Enterprise into one declarative component.

5.1 Composition

`<MarketsGrid>` composition

One React component that composes three packages.

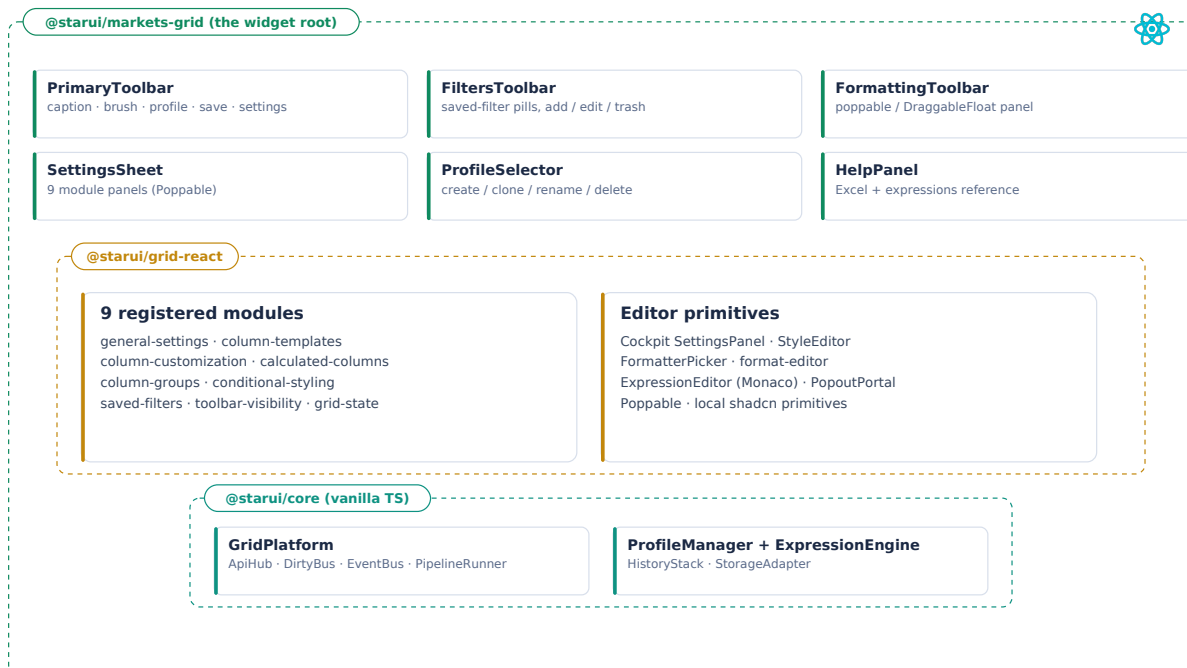


Figure 10: composition — chrome surfaces, customizer modules, core platform.

The diagram is dense — three layers worth pointing out:

- **The widget itself** (packages/react/widgets/markets-grid/) owns the chrome (PrimaryToolbar, FiltersToolbar, FormattingToolbar, SettingsSheet, ProfileSelector, HelpPanel), plus the StreamSafe floating-filter implementations (text / number / date).
- **@starui/grid-react** (packages/react/widgets/grid-react/) owns the modules (the 9 registered modules with their priority numbers — see §5.2), the Cockpit settings-panel primitives, the Monaco-based ExpressionEditor, the StyleEditor / FormatterPicker / format-editor primitives, and the PopoutPortal infrastructure.
- **@starui/core** owns the vanilla GridPlatform (with ApiHub, DirtyBus, EventBus, PipelineRunner, CssInjector, and topological sort by module priority), the ProfileManager, ExpressionEngine, and HistoryStack.

If you're adding a new band to an existing settings panel, you work in `grid-react`. If you're adding a new toolbar button, you work in `markets-grid`. If you're adding a new module *kind*, you author in `grid-react` and register it in `markets-grid`'s `DEFAULT_MODULES`.

5.2 The module pipeline

Every time the AG-Grid `columnDefs` or `gridOptions` are about to update, the platform runs the registered modules through a priority-sorted pipeline. Each module gets a chance to transform the defs and options before they reach `AgGridReact`.

Module pipeline

Modules transform `columnDefs` / `gridOptions` in priority order.

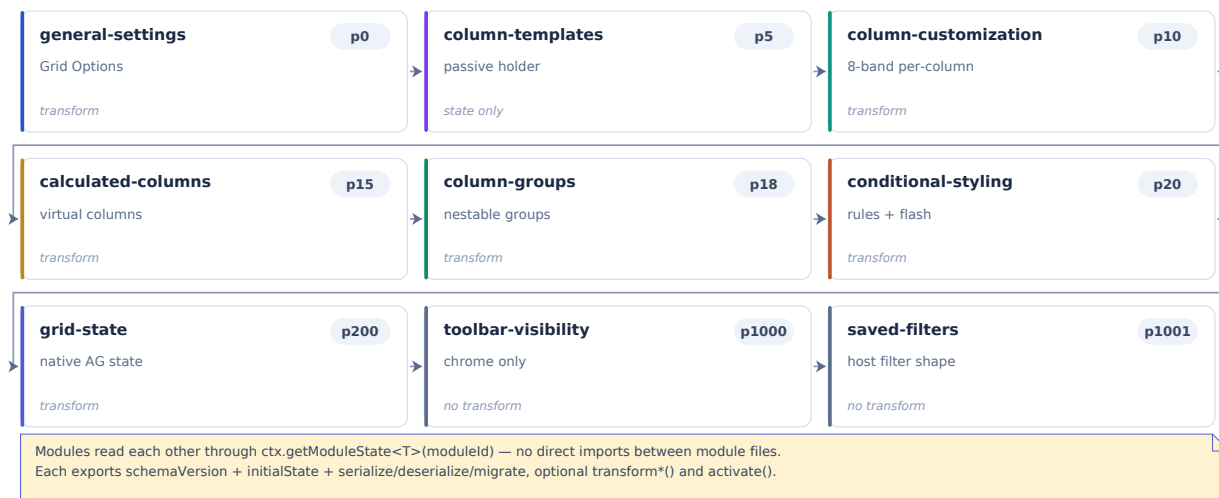


Figure 11: Module pipeline — modules sorted by priority, each transforming defs/options.

The 9 modules and their priorities are canonical:

general-settings	0	Grid Options (60 controls / 8 bands)
column-templates	5	Reusable override bundles
column-customization	10	Per-column overrides (8 bands)
calculated-columns	15	Virtual columns + cross-row aggregates
column-groups	18	Nestable header groups (depth 3)
conditional-styling	20	Expression-driven row/cell rules + flash
grid-state	200	Native AG state capture (Save only)
toolbar-visibility	1000	Host chrome layout
saved-filters	1001	Filter pills (host-defined shape)

Each module exports a common shape:

- `id`, `priority`, `schemaVersion`.
- `initialState`, `serialize / deserialize`, optional `migrate(raw, fromVersion)`.
- Optional `transformColumnDefs(ctx, defs)` and `transformGridOptions(ctx, opts)`.
- Optional `activate(api, ctx)` to subscribe to AG-Grid events.
- Optional `module.ListPane / module.EditorPane` React components for master-detail panels in the settings sheet.

Cross-module reads always go through `ctx.getModuleState<T>(moduleId)` — never direct imports between module files. That’s how `calculated-columns.transformColumnDefs` reads the template state without taking a hard dependency on `column-templates`.

Two priority subtleties worth knowing:

- **column-templates runs first** (priority 5) so that when `column-customization` (priority 10) walks the assignments, every template referenced by `templateIds[]` is already resolvable.
- **grid-state (priority 200) replays the native AG state on `onGridReady` and `profile:loaded`**, after all the schema modules have finished. This is the only module that *reads* AG-Grid state back into the profile snapshot (every other module just *transforms* the defs before AG-Grid sees them).

5.3 Storage

`MarketsGrid` accepts a storage prop that is a `StorageAdapterFactory((opts: { instanceId, appId, userId, gridId }) => StorageAdapter)`. Two factories ship with the platform:

1. **`createMarketsGridLocalStorageStorage()`** (from `@starui/markets-grid`) returns a branded factory backed by `LocalStorageBundleAdapter`. The bundle is keyed `markets-grid-bundle:${gridId}`; the active profile id mirrors at `gc-active-profile:${gridId}`. No `appId / userId` needed. This is what `apps/demo-apps/basic-starui-app` uses.
2. **`createConfigServiceStorage({ configManager, appId, userId })`** (from `@starui/config-service`) is the production factory backed by `ConfigService`. Requires `appId` and `userId`. The full profile set, active id, and grid-level data persist in one `AppConfigRow`. This is what `<HostedMarketsGrid>` wires up automatically.

If you don’t pass a storage prop, `MarketsGrid` warns once in dev mode and falls back to an in-memory adapter — useful for stories and demos, not for production.

6. Hosting in OpenFin

The reference OpenFin shell lives at `apps/markets-ui-react-reference`. It demonstrates every hosting pattern the platform supports — provider boot, dock-launched views, per-view identity, popouts, tab-rename round-trip, workspace save/restore.

6.1 The boot sequence

1. **OpenFin reads the manifest** (`platform.url` → `/platform/provider` route). `customSettings.useRest` and `customSettings.configServiceRestUrl` are read by `getConfigServiceRestUrlFromManifest()` from `@starui/openfin-platform/config`.
2. **The provider window boots** (hidden, frameless). It calls `initWorkspace()` which registers Home, Store, Dock, and Notifications, installs the `CustomActions` map (including the “Save Tab As...” rename action), and registers the `WorkspacePlatformOverride` for snapshot persistence and view-tab context menu injection.
3. **Background prefetch**. The provider opportunistically imports every tool-window route’s lazy chunk (`/blotters/marketsgrid`, `/dataproviders`, `/config-browser`, `/workspace-setup`, ...) so that when the dock launches them, they paint quickly.

OpenFin hosting — boot sequence

Manifest → hidden provider window → dock → per-view OpenFin Views.

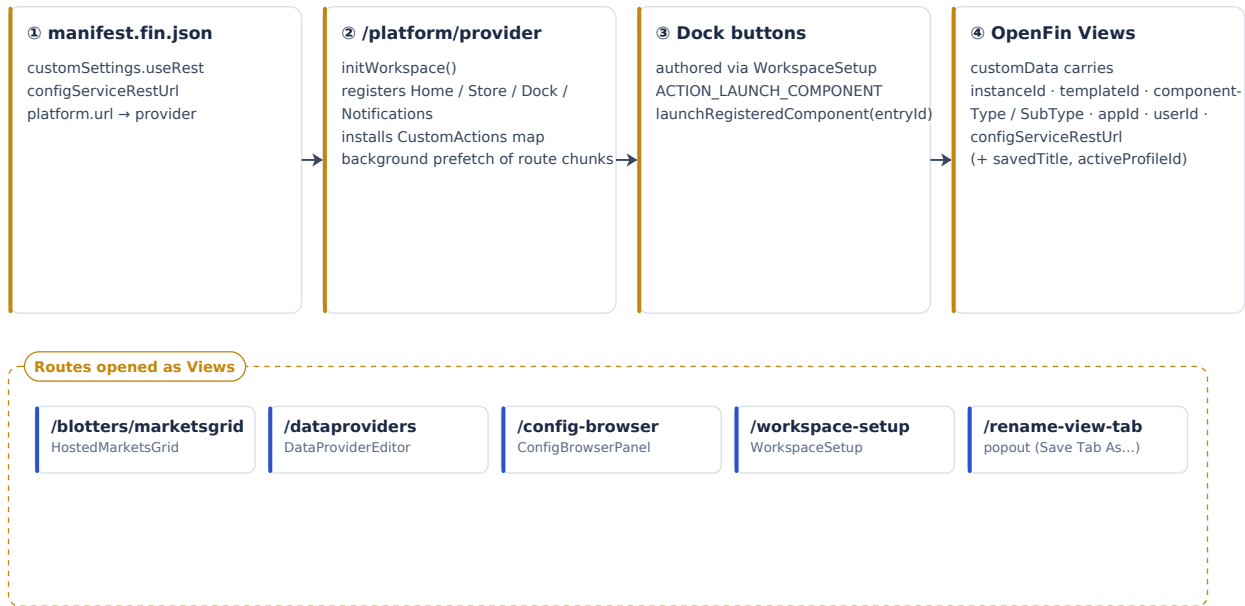


Figure 12: OpenFin hosting — manifest → provider window → dock → views.

4. **Dock buttons and dropdowns** (authored via the WorkspaceSetup editor) fire ACTION_LAUNCH_COMPONENT actions which call launchRegisteredComponent(entryId). That helper reads the component registry, finds the entry by id, and creates an OpenFin View (or Window if asWindow is set) with customData = { instanceId, templateId, componentType, componentSubType, appId, userId, configServiceRestUrl }.

Every view's customData is the platform's identity vector. It flows into OpenFinRuntime.resolveIdentity() → <HostWrapper> → useHost() → useHostedIdentity(), and any feature that needs to know "who am I, what app am I in, what backend should I talk to" reads it from there.

6.2 <HostedMarketsGrid> — the consolidated hosting wrapper

Before the consolidation, the boilerplate to host a MarketsGrid in OpenFin was a six-deep wrapper stack (BlottersMarketsGrid → HostedFeatureView → HostedComponent → BlotterGrid → MarketsGridContainer → MarketsGrid). All of that collapsed into one component:

The wrapper composes (top to bottom):

- **useHostedIdentity()** — resolves identity from OpenFin customData or browser fallback, builds the storage factory with RegisteredComponentMetadata injected.
- **useAgGridTheme()** — reads [data-theme] on <html>, returns the right themeQuartz-based theme.
- **useHostedView()** — umbrella composer over the six sub-hooks (useIab, useOpenFinChannel, useTabsHidden, useWorkspaceSaveEvent, useColorLinking, useFdc3Channel). The most important wiring is onWorkspaceSave → profiles.saveActiveProfile() — when the user clicks "Save Layout" in the OpenFin shell, MarketsGrid silently flushes the active profile through the same code path the toolbar Save button uses.
- **Provider stack:** <DataServicesProvider> (eager mode) → <FullBleed> → <ConfigMan-

<HostedMarketsGrid> — flat composition

A six-deep stack collapsed into one component. Hooks attach OpenFin lifecycle.

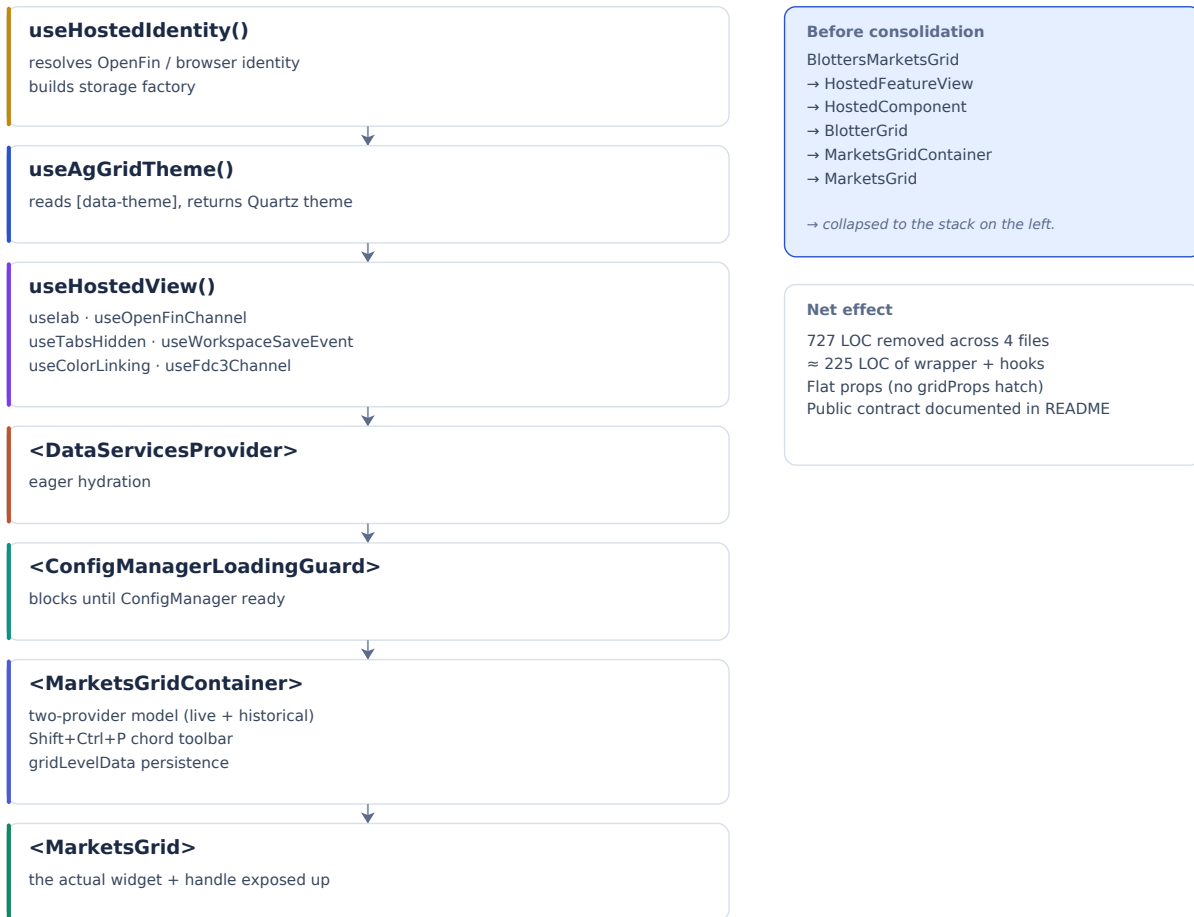


Figure 13: — single-component wrapper consuming hosted-view hooks.

agerLoadingGuard> → <MarketsGridContainer> (two-provider live/historical model) → <MarketsGrid> (the actual widget, with its handle exposed up).

The flat-props design (no gridProps escape hatch) is deliberate — the contract is documented in packages/react/widgets/widgets-react/src/hosted/README.md and is considered part of the public API.

6.3 Per-view active profile

In OpenFin, a trader can duplicate a MarketsGrid view tab and have each copy display a *different* saved profile of the same grid template. That works because ProfileManager reads its active-id from a pluggable ActiveIdSource rather than from localStorage directly.

createOpenFinViewProfileSource() (in @starui/markets-grid) returns a source that reads / writes activeProfileId on fin.me.getOptions().customData. Because OpenFin's Platform.getSnapshot() reads the same view options that updateOptions({ customData }) mutates, the per-view active id is captured into the workspace JSON automatically — no workspace-persistence.ts changes needed.

The read-priority chain:

OpenFin override → localStorage → reserved Default

Duplicate semantics: the duplicate view inherits the source view's customData (OpenFin's behaviour), then diverges as each user makes a switch. Exactly the desired UX.

6.4 “Save Tab As...” — surviving workspace restore

Renaming an OpenFin view tab sounds trivial — until you find out that View.updateOptions({ title }) is silently dropped at runtime (because title lives on the create-time ViewOptions shape, not on MutableViewOptions). The rename has to be done in two places at once, *and* survive a workspace restore:

Step by step:

1. **The context menu** (“Save Tab As...”) is injected by injectRenameMenuItem inside WorkspacePlatformOverride.openViewTabContextMenu.
2. **The custom action** opens a frameless popout (/rename-view-tab) with customData = { view, currentTitle }.
3. **The popout** is a shadcn Card + Input seeded with the current title. Enter or Save commits; Esc or Cancel closes.
4. **On commit, two writes against the target view:**
 - executeJavaScript: document.title = newTitle — the OpenFin workspace tabstrip updates immediately.
 - view.updateOptions({ customData: { ..., savedTitle } }) — the rename is captured by getSnapshot() into the workspace JSON.
5. **On the next workspace load**, OpenFinRuntime.applySavedViewTitle():
 - reads customData.savedTitle,
 - re-applies it to document.title,
 - sets a MutationObserver on <title> for 3 seconds post-boot so that the page's own useEffect(() => {document.title = ...}, []) can't clobber the restored title,
 - polls customData for live changes and re-applies on each transition (guarded by lastAppliedSavedTitle so unrelated customData mutations like activeProfileId don't trigger a re-apply).

"Save Tab As..." — survives workspace restore

A rename writes both `document.title` (live) and `customData.savedTitle` (snapshot).

① Right-click view tab

`WorkspacePlatformOverride.openViewTabContextMenu`
injects the menu item via `injectRenameMenuItem`.

② Open frameless popout

`ACTION_RENAME_VIEW_TAB` fires.
`openChildWindow(/rename-view-tab)` with
`customData = { view, currentTitle }`.

③ Popout form

shadcn Card + Input seeded with `currentTitle`.
Enter / Save commits, Esc / Cancel closes.

④ On confirm, two writes

`executeJavaScript: document.title = newTitle`
→ workspace tabstrip updates immediately
`view.updateOptions({ customData: { ..., savedTitle } })`
→ captured by `getSnapshot()`

⑤ Next workspace load

`OpenFinRuntime.applySavedViewTitle()`

- reads `customData.savedTitle`
- reapplies to `document.title`
- `MutationObserver` guards for 3 s post-boot

Why two writes? `View.updateOptions({ title })` is silently dropped at runtime —
``title`` lives on the create-time `ViewOptions` shape, not on `MutableViewOptions`.
Hence `document.title` (live tabstrip) + `customData.savedTitle` (workspace snapshot).

Figure 14: Save Tab As — rename flow, customData persistence, restore guard.

7. App compositions

The six apps in the repo each compose the libraries differently depending on what they're demonstrating. This is the cheat sheet for "where do I look when I want to see X in action?"

App compositions

Six apps, three patterns — pick the one closest to your use case.

apps/demo-react Primary E2E target  AppShell over BrowserRuntime URL view switcher (single · dashboard · depth · ...) Live ticking · showcase profile seed	apps/demo-configservice-react Multi-user demo  AppShell over BrowserRuntime Header user picker (dev1 / alice / bob) scopedActiveProfileKey per user ConfigBrowser via window.open popout	apps/markets-ui-react-reference OpenFin shell  OpenFinRuntime when isOpenFin() /platform/provider · /blotters/marketsgrid /dataproviders · /config-browser /workspace-setup · /rename-view-tab
apps/demo-apps/basic-starui-app Minimal example  createMarketsGridLocalStorageStorage() No AppShell · No ConfigService StatusStrip + ConfigInspector read the same localStorage bundle key	apps/demo-angular Angular dock-manager demo  32 standalone widget components DockManagerCoreComponent layouts PrimeNG (ChromaDeskPreset) Consumes only @starui/design-system today	apps/stomp-view-server Dev-time data source  Node + ws on :8081 Synthetic FI snapshots (~20k rows) /health · / metadata endpoints Used by demo MarketsGrid blotters

Figure 15: App composition — six apps, three patterns.

- **apps/demo-react** — the primary E2E target. Boots an `<AppShell>` over `BrowserRuntime` and the local `Dexie ConfigManager`. URL-driven view switcher (single | dashboard | depth | design-system | stockflux-blotter | fixture). Live ticking toggle persists via `gc-ticking localStorage`. First port of call when you want to reproduce a bug.
- **apps/demo-configservice-react** — multi-user demo. Same shell as `demo-react`, but the header has a user picker (dev1 / alice / bob) and `scopedActiveProfileKey` so per-user profile isolation is visible. `ConfigBrowser` is reachable via a `window.open`-backed popout (`?configBrowser=1`).
- **apps/markets-ui-react-reference** — the OpenFin shell. Runtime selection (`OpenFinRuntime` vs `BrowserRuntime`), REST endpoint from the manifest, the full `<DataServicesProvider>` → `<ConfigServiceProvider>` → `<HostWrapper>` stack in `<ViewRoutesLayout>`. This is the canonical reference for production hosting.
- **apps/demo-apps/basic-starui-app** — the minimal “just use `MarketsGrid`” example. No AppShell, no `ConfigService`, no `RuntimePort`. `createMarketsGridLocalStorageStorage()` + a custom `StatusStrip` + `ConfigInspector` that reads the same `marketsGridLocalStorage-BundleKey` the grid writes to. Useful as a template when you’re integrating `MarketsGrid` into a third-party app that doesn’t want the full provider stack.
- **apps/demo-angular** — Angular dock-manager demo with 32 standalone widgets and pre-built layouts (Trade / Prices / Risk / Market / Orders / Research / Design). **Currently consumes only @starui/design-system** from the workspace; the Angular providers (host-wrapper-angular, config-service-angular, data-services-angular, widgets-angular) exist but aren’t wired here yet — that’s the next Angular parity push.
- **apps/stomp-view-server** — a Node + ws server on localhost:8081 that serves synthetic fixed-income snapshots (~20k rows, configurable via `snapshot-rows` STOMP header) for the

demo blotters. Has /health and / HTTP endpoints. Not a view; not a library; just a dev-time data source.

8. Extension points

A few patterns worth knowing about when you're being asked to "add a new X". Each one is paired with the file you'd open first.

8.1 Adding a new grid module

```
packages/react/widgets/grid-react/src/modules/<your-module>/
├── index.ts           // exports your `Module<TState>` and registers priority
├── state.ts          // state shape, initial, serialize/deserialize, migrate
├── transforms.ts     // transformColumnDefs / transformGridOptions (optional)
├── YourPanel.tsx     // module.ListPane / module.EditorPane (optional)
└── *.test.ts
```

Then add the module to `DEFAULT_MODULES` in `packages/react/widgets/markets-grid/src/MarketsGrid.tsx` (or pass it via the `modules` prop if it's an app-specific extension).

If you need a settings-sheet entry, set `module.ListPane` and `module.EditorPane`. The `SettingsSheet` resolves them automatically.

8.2 Adding a new data-provider transport

```
packages/shared/services/data-services/src/runtime/providers/transport/<your-transport>.ts
// exports start<Your>(cfg, emit) → ProviderHandle
// and probe<Your>(cfg) for the editor's "Test connection" path
```

Wire it into `registerProvider` (`registry.ts`) and add a matching `ProviderConfig` shape in `@starui/shared-types`. Add a form to the v2 `DataProviderEditor` under `packages/react/widgets/widgets-react/src/v2/provider-editor/transport/`.

8.3 Adding a new tool window

If the tool sits inside the OpenFin shell, it's a React route in `apps/markets-ui-react-reference/src/main.tsx`, lazy-loaded, with a launcher action in `@starui/openfin-platform/launch.ts`. If it's also reusable as a non-OpenFin component, put the component in `packages/react/tools/<your-tool>-react/` and consume it from the route.

8.4 Adding a new admin action

`MarketsGrid` accepts `adminActions: AdminAction[]` — entries rendered in the settings-sheet Tools dropdown (Wrench icon). `createConfigBrowserAction` in `@starui/config-browser` is the reference implementation. Caller supplies just the launch callback; the platform decides whether to render it.

8.5 Adding a new framework-agnostic service

Goes in `packages/shared/services/`. The contract:

- Single export barrel (`src/index.ts`).
- No React or Angular imports.

- Peer-deps are either zero or other shared/ packages.
- Add a React provider in packages/react/providers/<your-service>-react/ and an Angular provider in packages/angular/providers/<your-service>-angular/ to plug it into hosts.

8.6 Replacing the runtime

Implement RuntimePort, pass your instance into <HostWrapper> (or provideHostWrapper() in Angular). Existing browser, OpenFin, and hypothetical Electron / WebView2 runtimes coexist without touching any consumer code. The interface is intentionally small; the same pattern works for testing — a MockRuntime is a valid extension point for unit tests of host-bound features.

9. StarUI MCP — scaffolding new apps

The StarUI Platform is **configuration-driven by design**. The <AppShell> provider stack, ConfigManager bootstrap, DataServices SharedWorker, <HostedMarketsGrid> wrapper, and every registered grid module all read their behaviour from typed configuration rather than from hard-coded wiring. That property is the foundation of a separate piece of the platform: an **MCP (Model Context Protocol) server** that lets developers scaffold entire StarUI applications through their AI assistant — without writing the boilerplate by hand.

StarUI MCP — scaffolding server

One prompt → fully-wired application. The framework's config-driven seams are the API surface.

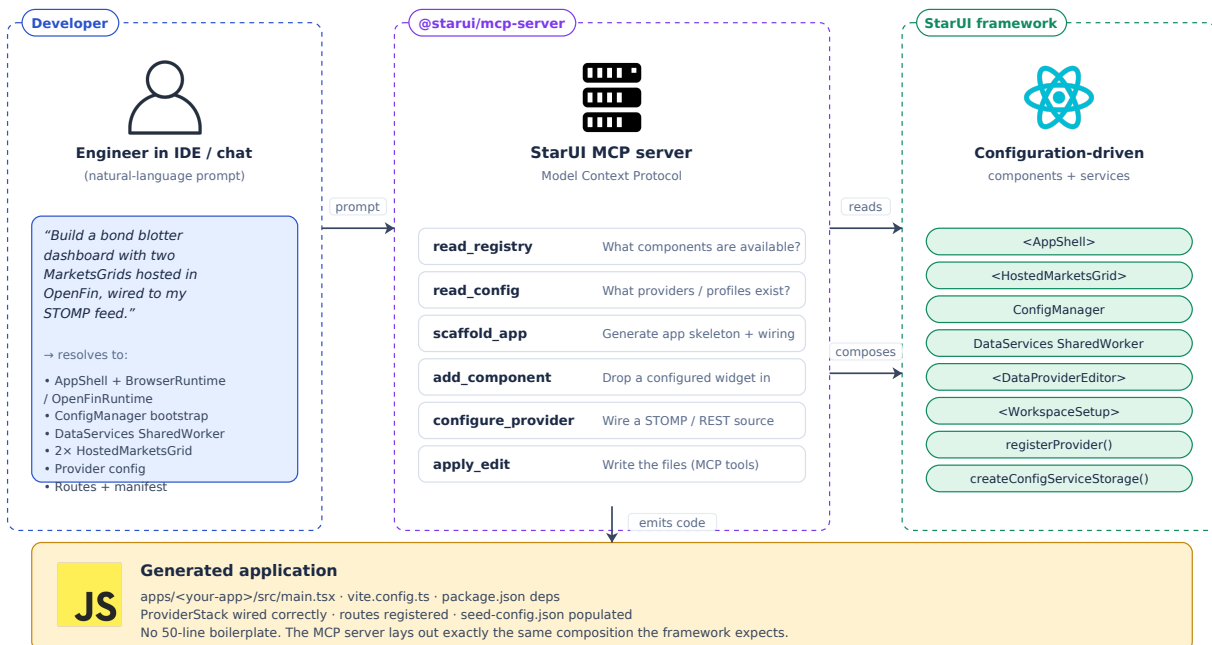


Figure 16: StarUI MCP — natural-language prompt to fully-wired application.

9.1 What the server does

The MCP server exposes a small set of tools that an AI assistant (Claude, an IDE chat panel, etc.) can call:

- **scaffold_app** — generate a new StarUI application skeleton from a one-line description. The output already has the right `<AppShell>` composition for the target runtime (browser vs OpenFin), a bootstrapped `ConfigManager`, the `DataServices SharedWorker` entry point, the storage factory chosen (`createMarketsGridLocalStorageStorage()` for minimal apps, `createConfigServiceStorage()` for full `ConfigService` apps), and the Vite config + dependency pins matching the canonical stack.
- **add_component** — drop a configured StarUI widget into an existing app. Knows the prop contract of `<HostedMarketsGrid>` / `<DataProviderEditor>` / `<WorkspaceSetup>` / `<Config-BrowserPanel>` / the v2 provider selector and never forgets `instanceId`, `storage`, `dataServices`, or `adminActions`.
- **configure_provider** — write a new `DataProviderConfig` row (stomp / rest / mock / appdata) into `ConfigService`. Resolves template tokens (`{{name.key}}` and `[token]`) correctly the first time.
- **read_registry + read_config** — query the live state of the component registry, dock configuration, `ConfigService` rows, and profile sets, so the assistant can answer “what providers are available?” or “add this registered component to the dock” with full awareness of what the user already has.
- **apply_edit** — write the file edits via standard MCP tooling (`package.json`, `vite.config.ts`, route declarations, `seed-config.json`, etc.).

9.2 Why this design

Configuration-driven framework + MCP server is the multiplicative effect. Each one alone is useful; together they collapse the cost of building a new StarUI app from days to minutes.

- **The framework’s seams are the API surface.** Every wiring decision the platform makes (which `RuntimePort`, which `StorageAdapterFactory`, which `ProfileManager.ActiveIdSource`, which `DEFAULT_MODULES` list, which `adminActions`) is exposed as a typed configuration option. The MCP server simply fills those options out. There is no parallel code-generation path the framework doesn’t already understand.
- **No boilerplate.** Consumers historically wrote 50–100 lines of provider stack, storage-factory closure, identity resolution, and manifest plumbing per app. The MCP server eliminates that. Importantly, it does so without inventing a new layer — the code it emits is the same composition `<HostedMarketsGrid>` already documents.
- **The API stays first-class.** Every prop, hook, and module the platform exposes is unchanged. The MCP server scaffolds the common path; engineers extend with code where they need custom behaviour (e.g. registering a new data provider transport via `registerProvider`, or pushing a new grid module into `DEFAULT_MODULES`). Customisation is additive, not a fight against the generator.
- **Time-to-first-blotter measured in minutes.** A new joiner who knows nothing about the framework can describe what they need in English and have a running OpenFin shell + `MarketsGrid` + `STOMP` data provider in their editor before the end of an onboarding session.

9.3 Status and roadmap

The server lives at `packages/shared/services/starui-mcp/` and is **under active development** — the directory currently holds a scaffold with empty `src/handlers/` and `src/tools/` trees (see §13 Known gaps for the current state). When complete it will be:

1. The **recommended entry point for new StarUI consumers** — added to the README’s “Getting started” as the default flow, with the manual provider-stack walkthrough kept as a fallback.

2. Wired through the same @starui/openfin-platform/config subpath the Config Browser uses, so it can read REST vs Dexie modes from manifest.fin.json and never gets out of sync with the deployed ConfigService backend.
 3. Versioned alongside the framework. A breaking change to <HostedMarketsGrid>'s prop contract is also a breaking change to add_component, and the server's compatibility matrix is part of the release notes.
-

10. Operations / dev workflow

A few things to know once you're committing code:

- **npm 10 workspaces. No pnpm, no yarn. No --legacy-peer-deps.** Plain npm ci should resolve cleanly; treat any ERESOLVE as a real bug to investigate (see CLAUDE.md for the policy).
 - **Turborepo orchestrates everything.** npm run build, npm run typecheck, npm test, npm run e2e all run via turbo against every package.
 - **rimraf dist && tsc is required** in every library's build script. Defeats a TS5055 collision with Turbo's cache restore.
 - **Hashed tarballs.** Reference apps (e.g. basic-starui-app) consume libraries via file: tarballs in libs/. Use npm run propagate -- <pkg> to repack — the script appends a short content-hash to each filename so npm's file: cache (which keys on path + integrity) can't serve a stale extraction.
 - **Token contract.** npm run check-ds runs three gates: check-ds-tokens (no hardcoded hex, no legacy --bn-* / --fi-* vars), check-design-system-deps (DS dep contract), check-react-apps-no-native-select (use shadcn <Select>, not native <select>).
 - **Doc lockstep.** Every feature add / change / remove updates docs/IMPLEMENTED_FEATURES.md (historical changelog) and docs/FEATURE_INVENTORY.md (current state) in the same commit.
-

11. Where to look first

If you're brand new to the codebase and have one hour:

1. Open docs/ARCHITECTURE.md for the layer diagram.
2. Skim docs/FEATURE_INVENTORY.md \$1-\$5 to see the package map.
3. Open packages/react/widgets/markets-grid/src/MarketsGrid.tsx — this is the widget root. Trace the props, the forwardRef handle, the storage factory, and the DEFAULT_MODULES import.
4. Open packages/react/widgets/grid-react/src/modules/general-settings/index.ts to see the simplest module shape.
5. Open apps/demo-react/src/main.tsx — see how a complete app wires up <AppShell>, the ConfigManager, and the storage factory.

If you're brand new and have a day:

6. Read this guide cover to cover.
 7. Run npm ci && npm run dev — apps/demo-react at http://localhost:5190.
 8. Open the SettingsSheet (gear icon top-right of the grid). Try editing a column under Column Settings, saving the profile, reloading the page, and confirming the changes persist.
 9. Run npm run dev:openfin to launch the OpenFin shell. Right-click a view tab → "Save Tab As..." to see the rename popout in action.
-

Generated 2026-05-17 from the live codebase. Diagram sources live in docs/architecture-figures/.